МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Вычислительная математика»

Тема: Метод хорд

Студент гр. 4342 _____ Озернов Е.Н.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы — изучить численное решение нелинейного уравнения f(x) = 0 методом хорд (ложного положения). В ходе работы требуется отделить корень на интервале [Left; Right], реализовать подпрограмму вычисления функции f(x) и алгоритм HORDA с заданной точностью Eps. Необходимо теоретически и экспериментально исследовать скорость сходимости метода, построив зависимость числа итераций от Eps. Также требуется оценить обусловленность метода, смоделировав ошибки в вычислении f(x) с помощью округления Round с параметром Delta и проанализировав влияние Delta на найденный корень.

Задание.

Пусть найден отрезок [a, b], на котором функция f (x) меняет знак. Для определенности положим f (a)>0, f (b)<0. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения f (x) = 0 принимаются значения c0, c1, . . . точек пересечения хорды с осью абсцисс, как это показано на рис.1.

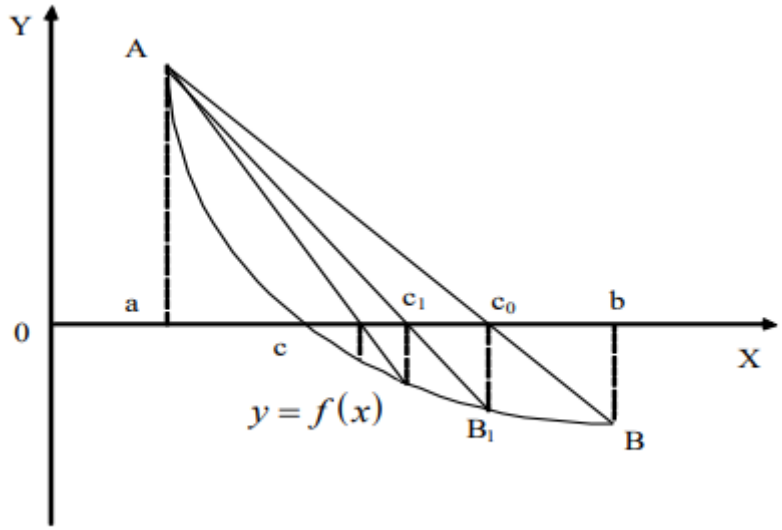

Рис. 1

Сначала находится уравнение хорды AB:

$$\frac{y-f(a)}{f(b)-f(a)}=\frac{x-a}{b-a}$$

Для точки пересечения ее с осью абсцисс (x=c0, y=0) получается уравнение

$$c_0=a-\frac{b-a}{f(b)-f(a)}f(a)$$

Далее сравниваются знаки величин f (a) и f (c0) и для рассматриваемого случая оказывается, что корень находится в интервале (a, c0), так как f (a) f (c0)<0. Отрезок [c0,b] отбрасывается. Следующая итерации состоит в определении нового приближения c1 как точки пересечения хорды AB1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение f (cn) не станет по модулю меньше заданного числа ε (см. подраздел 3.1).

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

В лабораторной работе №4 предлагается, используя программы - функции HORDA и Round из файла methods.cpp (файл заголовков metods.h, директория LIBR1), найти корень уравнения f (x) = 0 с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленности метода.

Для данной работы, как и для лабораторной работы №3 задаются индивидуальные варианты нелинейных уравнений (см. подраздел 3.6).

Порядок выполнения лабораторной работы №4:

1) Графически или аналитически отделить корень уравнения f (x) = 0 (т.е. найти отрезки [Left, Right], на которых функция f (x) удовлетворяет условиям применимости метода).

2) Составить подпрограмму - функцию вычисления функции f (x) , предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.

3) Составить головную программу, вычисляющую корень уравнения f (x) = 0 и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.

4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Вариант 16: f(x) = arcsin(2x/(1+x^2)) - e^(-x^2)

Выполнение работы.

1. Отделение (изоляция) корня уравнения $f(x) = 0$.

Рассматривается функция: f(x) = arcsin(2x / (1 + x^2)) − e^(−x^2).

Функция f(x) непрерывна на R, так как выражение 2x / (1 + x^2) при любых x принадлежит отрезку [−1; 1], поэтому arcsin(·) определена для всех x, а функция e^(−x^2) также непрерывна.

Для отделения корня используем теорему Коши (Больцано): если f(x) непрерывна на [Left; Right] и выполняется условие f(Left) * f(Right) < 0, то на интервале (Left; Right) существует хотя бы один корень уравнения f(x) = 0.

Проверим значения функции на концах интервала:

1. При x = 0:

f(0) = arcsin(0) − e^0 = 0 − 1 = −1 < 0.

2. При x = 0.5:

f(0.5) = arcsin(2*0.5 / (1 + 0.5^2)) − e^(−0.5^2) = arcsin(1 / 1.25) − e^(−0.25) = arcsin(0.8) − e^(−0.25) ≈ 0.9273 − 0.7788 ≈ 0.1485 > 0.

Так как f(0) < 0 и f(0.5) > 0, то: f(0) * f(0.5) < 0.

Следовательно, на отрезке [0; 0.5] существует корень уравнения f(x) = 0. Этот отрезок принимаем в качестве начального интервала для метода хорд: [Left; Right] = [0; 0.5].

2. Подпрограмма вычисления функции f(x) с учётом округления Round (Delta)

Для реализации метода хорд требуется подпрограмма, вычисляющая значение функции f(x) в заданной точке x. В данной лабораторной работе дополнительно необходимо предусмотреть моделирование ошибок исходных данных и вычислений, то есть округление значений функции с заданной точностью Delta с помощью подпрограммы Round.

Функция задаётся формулой:

f(x) = arcsin(2x / (1 + x^2)) − e^(−x^2).

Для вычисления используются стандартные математические функции asin(·) и exp(·). При вычислениях с типом double из-за округления возможно, что величина t = 2x / (1 + x^2) получится немного больше 1 или меньше −1, что может привести к ошибке при вычислении arcsin(t). Поэтому в подпрограмме аргумент арксинуса дополнительно ограничивают диапазоном [−1; 1]: если t > 1, то t = 1; если t < −1, то t = −1.

Моделирование ошибок выполняется следующим образом: вместо точного значения f(x) используется округлённое значение f_Delta(x) = Round(f(x), Delta).

Подпрограмма Round округляет переданное значение к ближайшему числу, кратному Delta. Если Delta = 0, округление не применяется и f_Delta(x) = f(x).

Таким образом, вычисление функции в работе выполняется в два шага: вычисление f(x) по формуле; округление результата: f_Delta(x) = Round(f(x), Delta).

Пример реализации в C++ (логика вычисления и округления):

```
double f(double x)
{
t = 2x / (1 + xx);
если t > 1, то t = 1;
если t < −1, то t = −1;
return asin(t) − exp(−x*x);
}
double f_Delta(double x, double Delta)
{
return Round( f(x), Delta );
}
```

Далее при работе метода хорд вместо f(x) на каждом шаге используются значения f_Delta(a), f_Delta(b), f_Delta(c), что позволяет исследовать влияние

параметра Delta на точность результата и устойчивость вычислительного процесса.

3. Головная программа: вычисление корня и обращение к f(x), HORDA, Round, индикация результатов.

Для выполнения лабораторной работы составляется головная программа (main), которая организует полный вычислительный процесс методом хорд: задаёт исходные параметры, вызывает подпрограмму вычисления функции f(x), запускает алгоритм HORDA, при необходимости включает округление значений функции через Round с параметром Delta и выводит результаты вычислений.
Головная программа включает следующие основные этапы.

1) Задание исходных данных и параметров расчёта

В программе задаются:

- подпрограмма вычисления функции:
  f(x) = arcsin(2x / (1 + x^2)) − e^(−x^2);
- интервал отделения корня из пункта 1:
  [Left; Right] = [0; 0.5];
- требуемая точность Eps (в дальнейшем используется набор значений Eps для исследования сходимости);
- ограничение nmax на число итераций (защита от отсутствия сходимости);
- параметр Delta для моделирования ошибок округления.
-

2) Обращение к подпрограмме f(x) и контроль корректности интервала

Перед запуском метода хорд вычисляются f(Left) и f(Right), чтобы убедиться в выполнении условия применимости метода: f(Left) * f(Right) < 0.

Если условие не выполняется, программа прекращает работу или выполняет автоматический поиск подходящего интервала.

3) Использование Round для моделирования ошибок (Delta)

В режиме исследования обусловленности программа должна имитировать вычисления с ограниченной точностью. Для этого применяется подпрограмма Round: f_Delta(x) = Round(f(x), Delta).

Именно значения f_Delta(x) используются внутри метода HORDA при вычислениях f(a), f(b) и f(c) на итерациях. При Delta = 0 округление не выполняется, и метод работает с исходными значениями функции.

4) Вызов процедуры HORDA (метод хорд)

Для нахождения корня уравнения f(x)=0 программа вызывает процедуру HORDA, передавая ей:

- функцию f(x);
- границы интервала [Left; Right];
- точность Eps;
- nmax;
- Delta (для Round).

На каждой итерации HORDA вычисляет приближение корня как точку пересечения хорды с осью Ox: c = a − f(a) * (b − a) / (f(b) − f(a)),
после чего выбирает новый отрезок [a; b], сохраняя смену знака. Итерации продолжаются до выполнения критерия остановки, например:
|f(c)| < Eps (и дополнительно может контролироваться малость интервала |b − a|).

5) Индикация (вывод) результатов

Головная программа выводит результаты на экран и сохраняет их в файлы для последующего анализа.

В консоль выводятся:

- найденный интервал [Left; Right];
- значения f(Left) и f(Right);
- найденное приближение корня x* (в методе хорд это последняя точка c);
- значение f(x*) (контроль близости к нулю);

- число итераций Iterations;
- признак успешной сходимости (ok).

Для исследования скорости сходимости формируется файл: iterations_vs_eps_horda.csv со столбцами: Eps; Iterations; Root; IntervalLen.

Для исследования чувствительности к округлению формируется файл: sensitivity_vs_delta_horda.csv со столбцами: Delta; Eps; Iterations; Root; AbsErrorToRef, где AbsErrorToRef = |x_Delta − x_ref|, а x_ref — опорное значение корня без округления.

Таким образом, головная программа объединяет вычисление f(x), метод хорд HORDA, округление Round (Delta) и вывод результатов, обеспечивая проведение экспериментов по исследованию сходимости и обусловленности метода.

4. Проведение вычислений. Теоретическое и экспериментальное исследование скорости сходимости и обусловленности метода хорд.

После реализации подпрограммы f(x), процедуры HORDA и функции Round выполняются вычисления по программе для анализа двух характеристик метода хорд: скорости сходимости и чувствительности к ошибкам исходных данных (обусловленности). Расчёты проводятся на отделённом интервале [Left; Right] = [0; 0.5], где выполняется условие применимости метода: f(Left) * f(Right) < 0.

1) Теоретические сведения о скорости сходимости метода хорд

Метод хорд строит последовательность приближений c0, c1, …, cn, … как точки пересечения хорд с осью Ox. На каждом шаге по текущим концам интервала a и b вычисляется новое приближение: c = a − f(a) * (b − a) / (f(b) − f(a)).

Затем выбирается новый отрезок [a; b], на котором сохраняется смена знака (то есть корень остаётся внутри отрезка). В отличие от метода бисекции, метод хорд обычно сходится быстрее, так как использует линейную аппроксимацию функции на текущем интервале. При благоприятных условиях

метод обладает линейной сходимостью и на практике требует меньшего числа итераций для достижения той же точности Eps.

Критерий остановки итераций в работе выбирается по малости значения функции в точке приближения: $|f(cn)| < Eps$.

Это означает, что искомое cn считается корнем с точностью Eps, так как f(cn) близко к нулю.

2) Экспериментальное исследование скорости сходимости

Для экспериментальной оценки скорости сходимости выполняется серия запусков метода HORDA при разных значениях точности Eps в диапазоне:

$Eps \in [0.1; 0.000001]$.

Для каждого значения Eps фиксируются:

- число итераций Iterations, необходимое для выполнения условия $|f(c)| < Eps$;
- найденное приближение корня Root;
- текущая длина интервала $IntervalLen = |b - a|$ (как дополнительный контроль).

Результаты автоматически сохраняются в файл:

iterations_vs_eps_horda.csv

со столбцами: Eps; Iterations; Root; IntervalLen.

По этим данным строится график зависимости числа итераций от Eps (Рисунок 1). По графику делается вывод о том, как быстро метод хорд достигает требуемой точности при уменьшении Eps, и сравнивается характер роста числа итераций с логарифмическим ростом, характерным для бисекции.

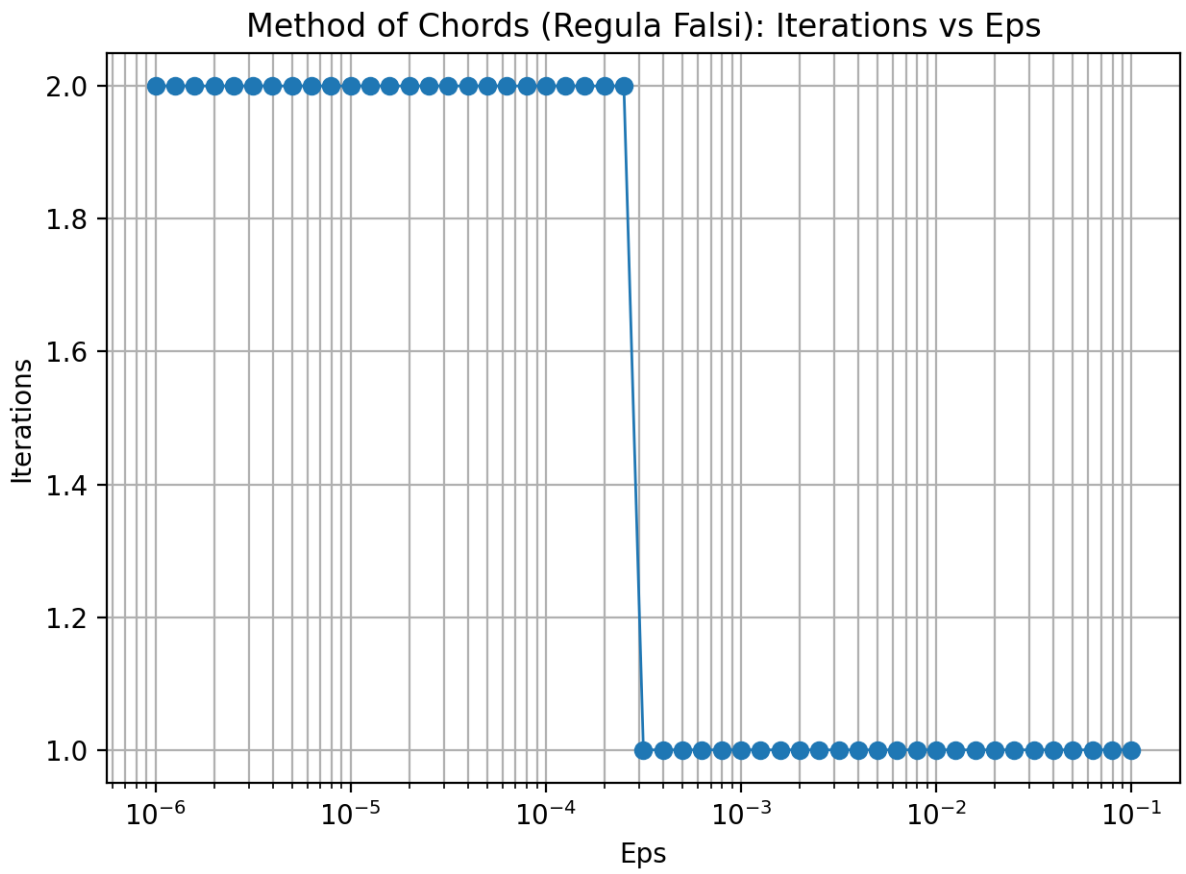

Рисунок 1 — График зависимости числа итераций метода хорд от точности Eps.

3) Теоретические сведения об обусловленности (чувствительности) метода

Обусловленность отражает, насколько результат вычислений зависит от ошибок в исходных данных и округлений при вычислении f(x). В методе хорд значение f(x) используется непосредственно в формуле:

c = a − f(a) * (b − a) / (f(b) − f(a)).

Поэтому искажения f(a) и f(b) могут изменять положение точки c, влиять на выбор нового интервала и, как следствие, изменять итоговое приближение корня и число итераций.

4) Экспериментальное исследование обусловленности с помощью Round

Чтобы смоделировать ошибки вычисления функции, применяется округление значений функции с параметром Delta:

f_Delta(x) = Round(f(x), Delta).

Для исследования выбирается фиксированная точность решения Eps (например, Eps = 0.000001). Далее:

1. Находится опорное значение корня без округления (Delta = 0) при высокой точности, оно принимается как эталон: x_ref.

2. Метод HORDA запускается при той же точности Eps, но с различными значениями Delta (например, Delta = 0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001).

3. Для каждого Delta фиксируются найденный корень x_Delta и число итераций Iterations.

4. Рассчитывается абсолютное отклонение от эталона:

AbsErrorToRef = |x_Delta − x_ref|.

Результаты записываются в файл:

sensitivity_vs_delta_horda.csv

со столбцами: Delta; Eps; Iterations; Root; AbsErrorToRef.

По этим данным строится график зависимости AbsErrorToRef от Delta (Рисунок 2). На основании графика делается вывод о влиянии округления: при уменьшении Delta ошибка AbsErrorToRef должна уменьшаться, то есть результат метода приближается к опорному значению x_ref; при больших Delta погрешность результата возрастает.

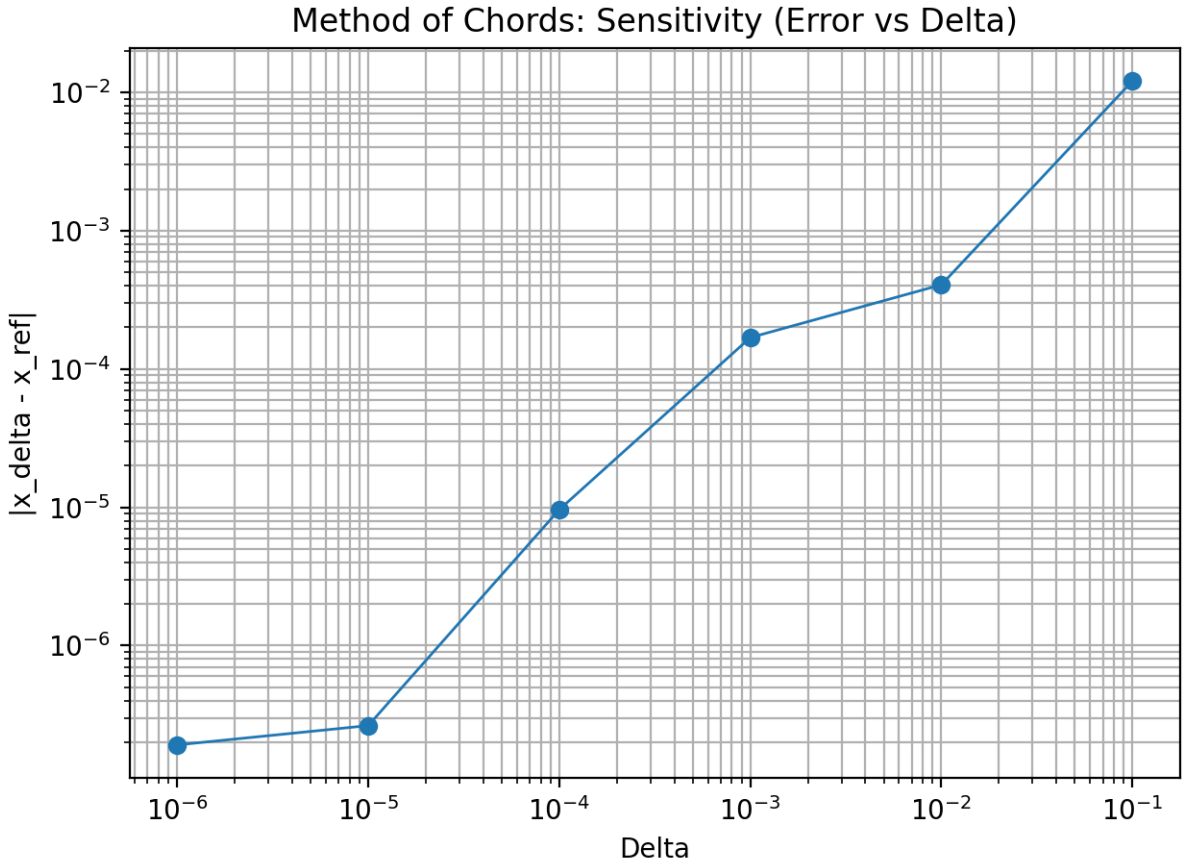

Рисунок 2 — Зависимость абсолютной ошибки |x_Delta − x_ref| от Delta (обусловленность метода хорд).

Таким образом, на данном этапе лабораторной работы метод хорд исследуется как теоретически (по его алгоритму и критериям остановки), так и экспериментально (по таблицам и графикам), что позволяет оценить скорость сходимости и устойчивость метода к ошибкам исходных данных.

Вывод.

В ходе лабораторной работы был изучен метод хорд для численного решения нелинейного уравнения f(x) = 0 на примере функции f(x) = arcsin(2x/(1 + x^2)) − e^(−x^2). Был выполнен этап отделения корня на отрезке [0; 0.5], реализованы подпрограмма вычисления f(x), алгоритм HORDA и моделирование ошибок вычисления функции с помощью округления Round с параметром Delta. Проведены вычисления при различных значениях точности Eps в диапазоне от 0.1 до 0.000001 и экспериментально исследована скорость сходимости, результаты представлены в виде таблицы и графика зависимости числа итераций от Eps. Также исследована обусловленность метода: при увеличении Delta округление сильнее влияет на значения функции и может заметно изменять найденное приближение корня, а при уменьшении Delta результат приближается к опорному значению, что подтверждает устойчивость метода при достаточно малых погрешностях.

ПРИЛОЖЕНИЕ А

Название файла: methods.h

```
#pragma once
#include <functional>

struct HordaResult {
    double x;        // приближение корня
    double a, b;     // текущий отрезок (сохраняем "скобку")
    int iters;       // число итераций
    bool ok;         // успешно ли сошлось
};

double Round(double value, double Delta);

// Метод хорд (regula falsi):
// f      - функция
// Left,Right - начальный отрезок, где f(Left)*f(Right)<0
// Eps    - точность
// nmax   - лимит итераций
// Delta  - округление значений f(...) через Round (0 => без округления)
HordaResult HORDA(const std::function<double(double)>& f,
                  double Left, double Right,
                  double Eps,
                  int nmax = 1000000,
                  double Delta = 0.0);
```

Название файла: methods.cpp

```
#include "methods.h"
#include <cmath>
#include <limits>

double Round(double value, double Delta) {
    if (Delta <= 0.0) return value;
    return std::round(value / Delta) * Delta;
}

HordaResult HORDA(const std::function<double(double)>& f,
                  double Left, double Right,
                  double Eps,
                  int nmax,
                  double Delta) {
    HordaResult res{};
    res.a = Left;
    res.b = Right;
    res.iters = 0;
    res.ok = false;
    res.x = std::numeric_limits<double>::quiet_NaN();

    double a = Left, b = Right;
    double fa = Round(f(a), Delta);
    double fb = Round(f(b), Delta);

    if (!(fa * fb < 0.0)) {
        return res; // нет смены знака
    }
```

```
    for (int n = 1; n <= nmax; ++n) {
        // формула точки пересечения хорды с осью X:
        // c = a - f(a)*(b-a)/(f(b)-f(a))
        double denom = (fb - fa);
        if (denom == 0.0) { // защита
            res.x = 0.5 * (a + b);
            res.a = a; res.b = b; res.iters = n; res.ok = false;
            return res;
        }

        double c = a - fa * (b - a) / denom;
        double fc = Round(f(c), Delta);

        // условия остановки
        if (std::abs(fc) < Eps) {
            res.x = c; res.a = a; res.b = b; res.iters = n; res.ok =
true;
            return res;
        }
        if (std::abs(b - a) < 2.0 * Eps) {
            res.x = c; res.a = a; res.b = b; res.iters = n; res.ok =
true;
            return res;
        }

        // сохраняем отрезок с корнем (bracketing)
        if (fa * fc < 0.0) {
            b = c;
            fb = fc;
        } else {
            a = c;
            fa = fc;
        }
    }

    res.x = 0.5 * (a + b);
    res.a = a; res.b = b; res.iters = nmax; res.ok = false;
    return res;
}
```

Название файла: main.cpp

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <vector>
#include <fstream>
#include "methods.h"

// f(x) = arcsin(2x/(1+x^2)) - e^(-x^2)
double f(double x) {
    double t = (2.0 * x) / (1.0 + x * x);

    // защита от погрешностей double
    if (t > 1.0) t = 1.0;
    if (t < -1.0) t = -1.0;
```

```
    return std::asin(t) - std::exp(-x * x);
}

// Авто-отделение корня: ищем смену знака на сетке
bool find_bracket(double x_min, double x_max, double step, double &Left,
double &Right) {
    double prev = x_min;
    double fprev = f(prev);

    for (double x = x_min + step; x <= x_max; x += step) {
        double fx = f(x);

        if (fprev == 0.0) { Left = prev; Right = prev; return true; }
        if (fprev * fx < 0.0) { Left = prev; Right = x; return true; }

        prev = x;
        fprev = fx;
    }
    return false;
}

int main() {
    std::cout << std::fixed << std::setprecision(12);

    // 1) Отделение корня
    double Left = 0.0, Right = 0.0;
    if (!find_bracket(-5.0, 5.0, 0.1, Left, Right)) {
        std::cerr << "Не удалось отделить корень на [-5,5] с шагом 0.1\n";
        return 1;
    }

    std::cout << "Отделение корня (авто): [" << Left << "; " << Right <<
"]\n";
     std::cout << "f(Left)=" << f(Left) << ", f(Right)=" << f(Right) <<
"\n\n";

    // 2) Опорный (reference) корень: высокая точность, без округления
    auto ref = HORDA(f, Left, Right, 1e-14, 2000000, 0.0);
    if (!ref.ok) {
       std::cerr << "Не удалось получить reference-корень методом хорд\n";
        return 2;
    }
    std::cout << "Reference root (Eps=1e-14, Delta=0): x=" << ref.x
              << "  iters=" << ref.iters << "\n\n";

    // 3) Скорость сходимости: Iterations vs Eps (0.1 .. 1e-6)
    std::ofstream feps("iterations_vs_eps_horda.csv");
    feps << "Eps;Iterations;Root;IntervalLen\n";

    for (int i = 0; i <= 50; ++i) {
        double t = i / 50.0;          // 0..1
        double p = -1.0 - 5.0 * t;    // -1..-6
        double Eps = std::pow(10.0, p);

        auto r = HORDA(f, Left, Right, Eps, 2000000, 0.0);
        double len = std::abs(r.b - r.a);

        feps << std::setprecision(16)
```

```
            << Eps << ";" << r.iters << ";" << r.x << ";" << len << "\n";
    }
    feps.close();
    std::cout << "CSV сохранён: iterations_vs_eps_horda.csv\n";

    // 4) Обусловленность: влияние Delta (округление значений функции)
    const double EpsFix = 1e-6;
    std::vector<double> deltas = {0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-
6};

    std::ofstream fdel("sensitivity_vs_delta_horda.csv");
    fdel << "Delta;Eps;Iterations;Root;AbsErrorToRef\n";

    for (double Delta : deltas) {
        auto r = HORDA(f, Left, Right, EpsFix, 2000000, Delta);
        double err = std::abs(r.x - ref.x);

        fdel << std::setprecision(16)
             << Delta << ";" << EpsFix << ";" << r.iters << ";" << r.x <<
";" << err << "\n";
    }
    fdel.close();
    std::cout << "CSV сохранён: sensitivity_vs_delta_horda.csv\n";

    return 0;
}
```

Название файла: plot_graphs.py

```
import pandas as pd
import matplotlib.pyplot as plt

# 1) Iterations vs Eps
eps_df = pd.read_csv("iterations_vs_eps_horda.csv", sep=";")

plt.figure()
plt.plot(eps_df["Eps"], eps_df["Iterations"], marker="o", linewidth=1)
plt.xscale("log")
plt.xlabel("Eps")
plt.ylabel("Iterations")
plt.title("Method of Chords (Regula Falsi): Iterations vs Eps")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("iterations_vs_eps_horda.png", dpi=200)
plt.show()

# 2) Error vs Delta (убираем Delta=0 для лог-шкалы по X)
delta_df = pd.read_csv("sensitivity_vs_delta_horda.csv", sep=";")
delta_df_nonzero = delta_df[delta_df["Delta"] > 0].copy()

plt.figure()
plt.plot(delta_df_nonzero["Delta"], delta_df_nonzero["AbsErrorToRef"],
marker="o", linewidth=1)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Delta")
plt.ylabel("|x_delta - x_ref|")
plt.title("Method of Chords: Sensitivity (Error vs Delta)")
```

```
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("sensitivity_vs_delta_horda.png", dpi=200)
plt.show()
```